First Interim Report: Word Embeddings via Matrix Factorization

Mariia Hamaniuk Mykhailo Rykhalskyi Ivan Zarytskyi

Introduction

Natural Language Processing enables applications such as machine translation, search engines, and LLMs. A core challenge is representing words mathematically for computers.

The classical approach - one-hot encoding, a binary vector of length V with a single 1 - fails to capture meaning. It treats every word as equally distant from every other, with no context. Word embeddings solve this by mapping each word to a dense, low-dimensional vector so that semantically related words are geometrically close.

Our project demonstrates that meaningful semantic structure can be extracted from raw text using nothing but counting and Linear Algebra. The method is theoretically grounded: optimality of the decomposition is guaranteed by the Eckart-Young theorem, making it a rigorous, interpretable application of Linear Algebra.

Aim: Build word vector representations from scratch using truncated SVD applied to a PPMI-weighted co-occurrence matrix, and analyze the geometric properties of the resulting embeddings.

Tasks:

1. Construct co-occurrence matrix $M \in \mathbb{R}^{V \times V}$ from the Text8 corpus (V=10,000, window 5).
2. Apply PPMI to remove frequency bias (words like "the", "a" etc. have high count but meaningless).
3. Compute truncated SVD and extract k-dimensional word vectors.
4. Evaluate via nearest-neighbour queries and word analogy tests.
5. Compare embedding variants U_k, $U_k\Sigma_k$, $U_k\Sigma_k^{1/2}$ on a benchmark.
6. Analyze the singular value spectrum to justify the choice of k.

Methods

Literature Review and Method Choice

Count-based methods. Latent Semantic Analysis [1] first applied SVD to term-document matrices for semantic similarity. Turney and Pantel [2] survey vector space models and formalize the distributional hypothesis: words appearing in similar contexts have similar meanings.

PPMI weighting. Bullinaria and Levy [3] showed PPMI-weighted matrices outperform raw counts on similarity benchmarks. PPMI discards spurious associations caused by high-frequency words (e.g. "the"):

$$\mathrm{PPMI}(i,j) = \max\left(\log_2 \frac{P(i,j)}{P(i)\,P(j)},\ 0\right)$$

where $P(i,j)$ is the joint co-occurrence probability and $P(i), P(j)$ are marginal probabilities.

Neural alternatives and the SVD connection. Word2Vec [4] and GloVe [5] achieve state-of-the-art analogy performance via neural training. However, Levy and Goldberg [6] proved that Word2Vec with negative sampling implicitly factorizes a matrix equivalent to PPMI. Our SVD approach is therefore *theoretically equivalent* to the neural method while remaining fully interpretable.

Our choice. We use truncated SVD on a PPMI matrix because (a) it directly applies LA concepts from the course (SVD, Eckart–Young, change of basis), (b) every step is transparent and interpretable, and (c) its optimality is mathematically proven.

Implementation Pseudocode

```
# 1. Co-occurrence matrix
for each word i in corpus:
    for each word j in context window (w=5):
        M[i][j] += 1

# 2. PPMI weighting
Total = sum(M)
for each (i,j) where M[i][j] > 0:
    pmi = log2(M[i][j]*Total / (rowsum[i]*colsum[j]))
    M_ppmi[i][j] = max(pmi, 0)

# 3. Truncated SVD (sparse, efficient)
U, S, Vt = svds(M_ppmi, k=300)
sort descending by S

# 4. Word vectors
E = U * sqrt(S)          # shape: V x k

# 5. Nearest neighbours
E_norm = normalize rows of E
scores = E_norm @ E_norm[query]
return top-n words by score

# 6. Analogy  (a:b :: c:?)
query = E[c] + E[b] - E[a]
return nearest neighbour of query
```

Results

Dataset: Text8 - 17 million tokens of cleaned Wikipedia text (publicly available). Vocabulary capped at V=10,000 most frequent words. No additional data collection needed.

Qualitative evaluation: nearest-neighbour check (king→{queen, prince}, dog→{cat, pet}) and analogy arithmetic (king−man+woman≈queen).

Quantitative evaluation: Google Analogy Benchmark [4] (19,544 questions); accuracy measured across $k \in \{50, 100, 200, 300, 500\}$ and all three embedding variants. t-SNE projection of 500 words to 2D to verify semantic cluster formation.

Baseline and model comparisons. To contextualize results, our SVD embeddings will be evaluated alongside:

- **Cosine similarity on raw co-occurrence counts** (no PPMI, no SVD) - naive baseline to isolate the contribution of PPMI weighting and dimensionality reduction.
- **GloVe** [5] (pre-trained, 300d, trained on Wikipedia) - count-based neural alternative directly comparable in data regime.
- **Word2Vec Skip-Gram with Negative Sampling** [4] (pre-trained, 300d) - neural SoTA reference and performance upper bound.

This comparison will clarify how much performance is lost relative to neural methods, and whether PPMI+SVD reaches competitive cosine similarity scores on semantic clusters despite its simplicity.

Challenges

Algorithmic limitations in our scenario.

- **Inability to capture word order.** The co-occurrence window is symmetric and bag-of-words: "dog bites man" and "man bites dog" produce identical counts. Syntactic analogies that depend on ordering (e.g. verb–subject relations) are systematically harder for our model than for order-aware neural architectures.
- **No subword information.** SVD operates on whole-word types. Rare inflected forms ("running", "runs") receive separate, poorly estimated vectors or fall outside the vocabulary entirely, whereas methods such as fastText decompose words into character n-grams and generalise to unseen forms.
- **PPMI sensitivity to corpus size.** PPMI estimates are unreliable for low-count co-occurrences. With V=10,000 and a window of 5 on 17M tokens, a significant fraction of matrix entries are zero or near-zero, potentially adding noise to the resulting singular vectors.
- **Global optimality vs. task optimality.** Eckart–Young guarantees the best rank-k approximation in Frobenius norm, but this reconstruction objective is not directly aligned with the analogy or similarity tasks we evaluate. Neural methods optimise task-relevant objectives end-to-end, partially explaining the performance gap.

Implementation challenges.

- **Memory.** A 10,000 $\times$ 10,000 dense matrix requires $\sim$800 MB. We use `scipy.sparse` throughout.
- **SVD speed.** Full SVD is infeasible; we use `scipy.sparse.linalg.svds` (Lanczos algorithm) for top-k singular vectors only.
- **Vocabulary cutoff.** Words outside the top 10,000 are excluded, potentially reducing benchmark coverage. We will try V=20,000 if hardware allows.
- **Benchmark gaps.** Some analogy pairs involve out-of-vocabulary words. We will report accuracy over both full and in-vocabulary subsets.

Plan and Timeline

Week	Task
1–2	Corpus preprocessing; sparse co-occurrence matrix
3	PPMI weighting; unit tests on toy vocabulary
4	Truncated SVD; word vector extraction
5	Nearest-neighbour evaluation; analogy benchmark
6	Comparison of k values and embedding variants
7	t-SNE; singular value spectrum analysis
8	Final report and presentation preparation

References

[1] S. Deerwester et al. (1990). Indexing by latent semantic analysis. JASIS.

[2] P. Turney & P. Pantel (2010). From frequency to meaning. JAIR, 37.

[3] J. Bullinaria & J. Levy (2007). Extracting semantic representations from word co-occurrence statistics. Behavior Research Methods, 39(3).

[4] T. Mikolov et al. (2013). Efficient estimation of word representations in vector space. ICLR.

[5] J. Pennington et al. (2014). GloVe: Global vectors for word representation. EMNLP.

[6] O. Levy & Y. Goldberg (2014). Neural word embedding as implicit matrix factorization. NeurIPS.